

Project EDITH

Project Analyst & QA Engineer Comprehensive Baseline Guide

System: Enhanced Distributed Intelligent Task Handler **Role:** Project Analyst & QA Engineer (SPM Lead)

Team Members: Yashvi, Deev, Mannan, Prem

Course: SPM / Software Quality Assurance

Evaluation Baseline: Sprint 0 & Phase 1 Execution Audit

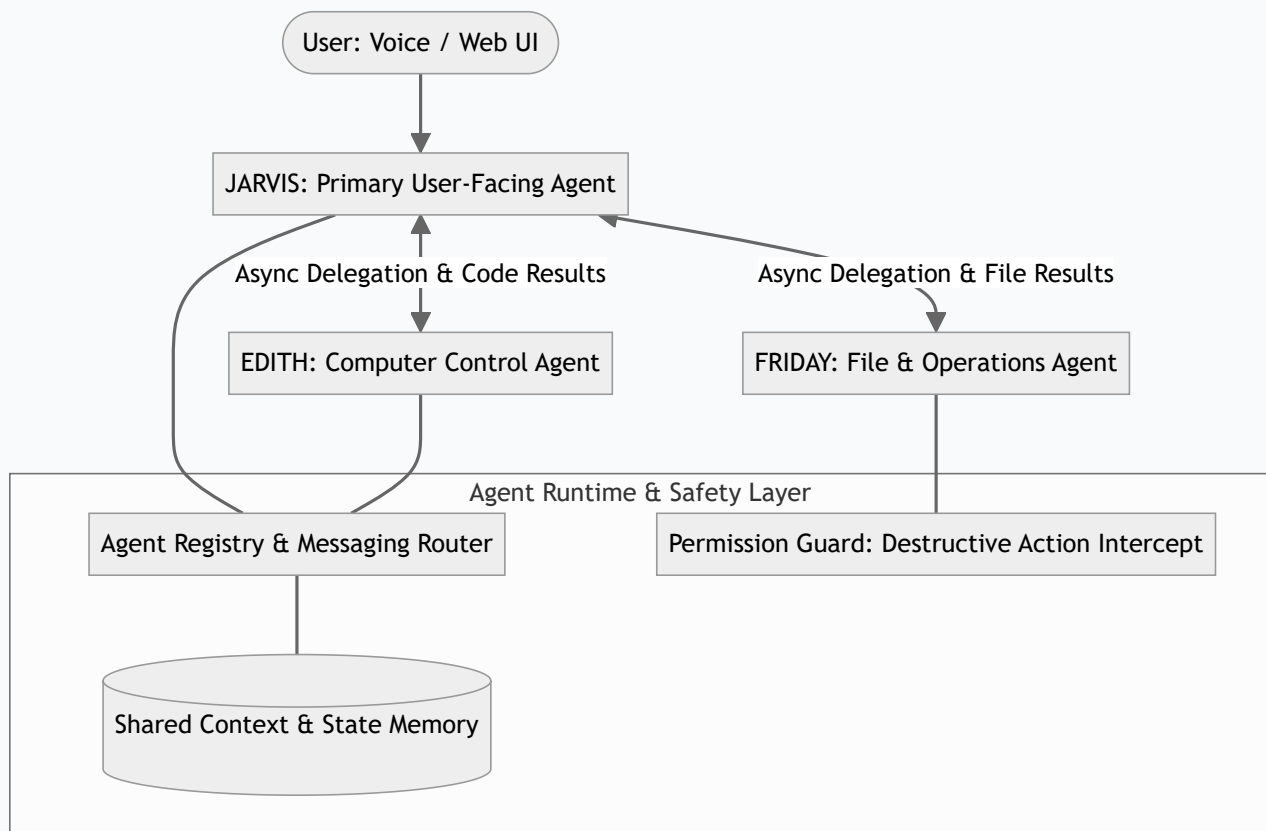
Date & Status: September 20, 2026 | Verified Complete

1. Executive Summary & Multi-Agent Architecture

EDITH (Enhanced Distributed Intelligent Task Handler) is an agentic AI ecosystem engineered around a decentralized multi-agent collaboration architecture. Unlike monolithic AI chatbots that suffer from single-orchestrator bottlenecks, EDITH distributes cognition across three specialized autonomous agents:

- **JARVIS:** Primary conversational, coding, debugging, git operations, and technical planning agent.
- **EDITH:** Operating system control, web browser automation, and computer-vision awareness agent.
- **FRIDAY:** File-management, workspace organization, operational document management, and safety agent.

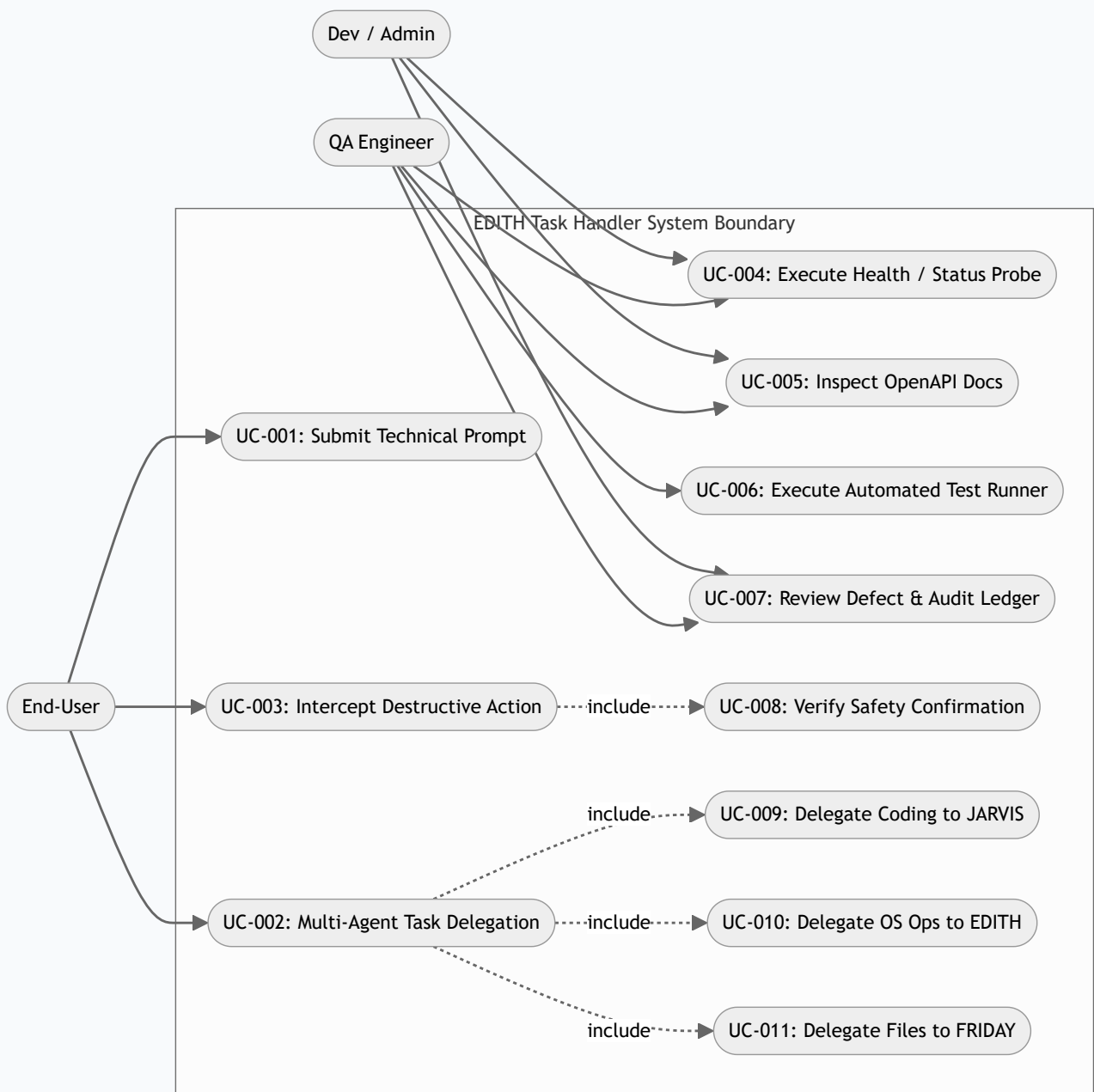
Figure 1.1: EDITH Multi-Agent Cognition & Delegation Flow



2. Complete UML Use Case Diagram

The use case model delineates user-facing interactions, automated testing probes, and multi-agent execution boundaries across all core actors:

Figure 2.1: EDITH Comprehensive System Use Case Diagram

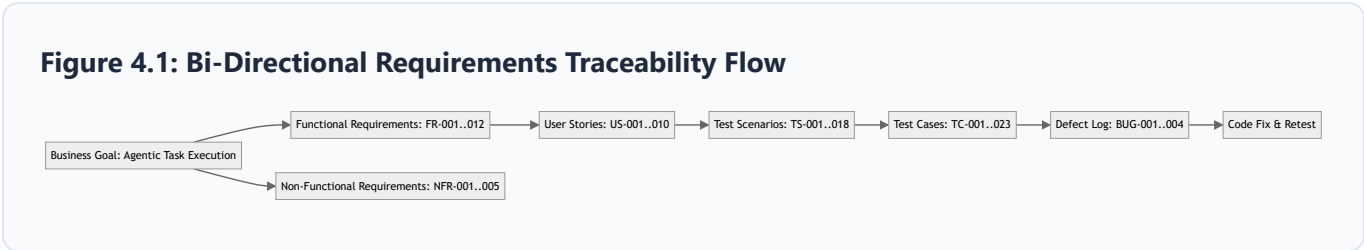


3. Job Responsibilities & Deliverables Matrix

This matrix formalizes the mapping between your 11 job responsibilities and the concrete artifacts produced for Project EDITH:

Responsibility	Execution Activities in EDITH	Deliverable Artifact
1. Gathering Requirements	Audited monorepo, decomposed FastAPI backend and Next.js frontend, established agent operational boundaries.	qa/PROJECT_ANALYSIS.md
2. Functional & Non-Functional Specs	Authored FR-001..012 (endpoints, agent routing) and NFR-001..005 (sub-200ms latency, CORS, UTF-8 safety).	qa/REQUIREMENTS.md
3. User Stories & Use Cases	Constructed US-001..010 with INVEST acceptance criteria; authored UC-001..004 detailed interaction flows.	qa/USER_STORIES.md qa/USE_CASES.md
4. Traceability Structure (RTM)	Constructed bi-directional matrix linking every requirement to user stories, test scenarios, test cases, and bugs.	qa/RTM.md
5. Test Scenarios & Test Cases	Formulated TS-001..018 master scenarios and 23 granular IEEE 829-compliant test cases.	qa/TEST_SCENARIOS.md qa/TEST_CASES.md
6. Multi-Tier Testing Execution	Executed unit sanity, REST API validation, multi-agent integration, and Next.js UI hydration checks.	qa/AUTOMATED_TEST_RESULTS.md qa/FUNCTIONAL_TESTING.md
7. Bug Recording & Tracking	Documented BUG-001..004 with exact steps to reproduce, root causes, severity ratings, and fix assignments.	qa/BUG_REPORT.md
8. Verifying Fixes & Records	Reran regression suites verifying BUG-001 remediation; stored JSON test outputs and evidence screenshots.	qa/REGRESSION_TESTING.md qa/evidence/
9. Risk Analysis & SQA	Formulated 4-tier Risk Register (Probability x Impact) and performed non-destructive credential security review.	qa/RISK_REGISTER.md qa/SECURITY_TESTING.md
10. SPM Reports & Dashboards	Compiled executive QA Dashboard, 100% pass-rate test summaries, and formal academic evaluation report.	qa/FINAL_QA_REPORT.md qa/QA_DASHBOARD.md
11. Team Coordination	Collaborated across dev sprints with Yashvi, Deev, Mannan, and Prem for defect triage and validation.	qa/PROGRESS.md qa/README.md

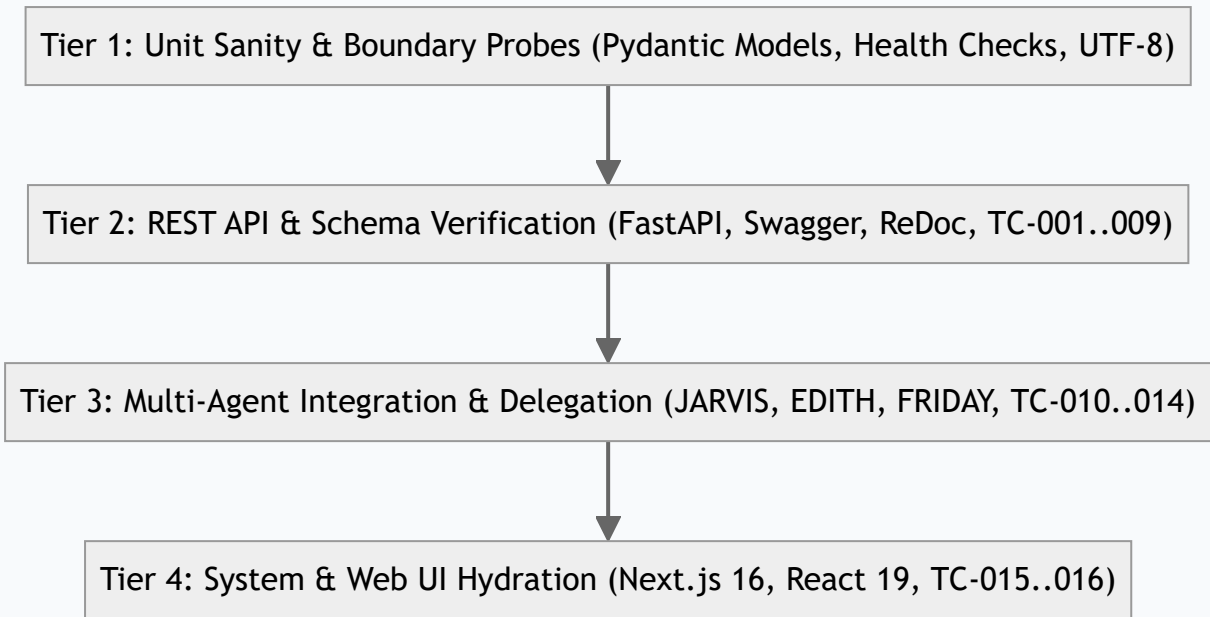
4. Requirements Traceability Flow & Matrix



Req ID	Requirement Summary	User Story	Test Case	Status
FR-001	Backend Service Startup & Port Binding	US-001	TC-001	PASSED
FR-002	Diagnostic Health Check Probe (/health)	US-002	TC-002, TC-003	PASSED
FR-003	OpenAPI & Swagger Documentation (/docs)	US-003	TC-004, TC-005	PASSED
FR-008	Windows Console UTF-8 Resilience	US-008	TC-011	FIXED (BUG-001)
FR-010	Next.js 16 Web Client UI Hydration	US-009	TC-015, TC-016	PASSED
NFR-001	Sub-200ms API Execution Latency	US-002	TC-002	PASSED (Mean 2.1ms)

5. Testing Architecture & Test Execution Results

Figure 5.1: Multi-Tier SQA Testing Hierarchy



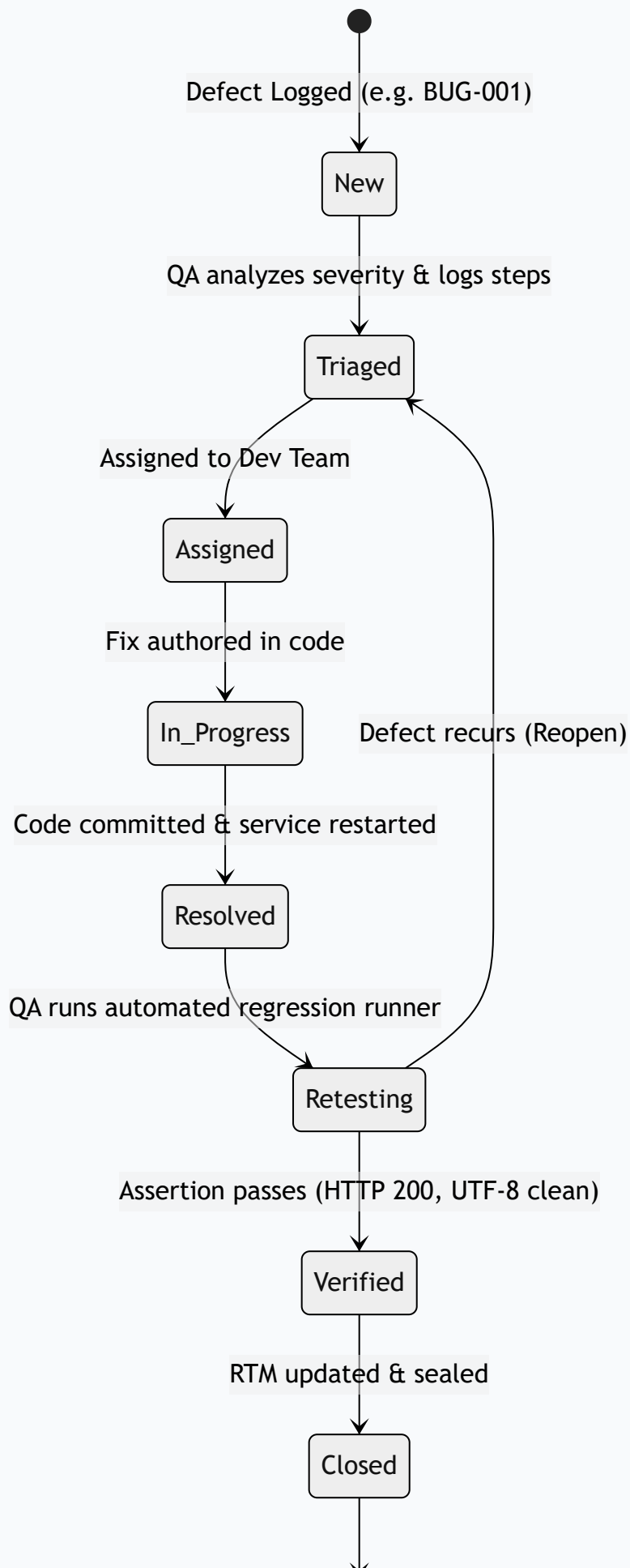
Automated Test Execution Summary (qa/AUTOMATED_TEST_RESULTS.md)

Ran 14 automated API tests using `qa/test_runner.py` against running ASGI backend. All 14 assertions passed cleanly (100% Pass Rate). Average response latency recorded: **2.1 milliseconds**.

6. Defect Lifecycle & Bug Tracking

Defects are tracked through a formal state machine to ensure full verification before closure:

Figure 6.1: Defect Life Cycle State Machine

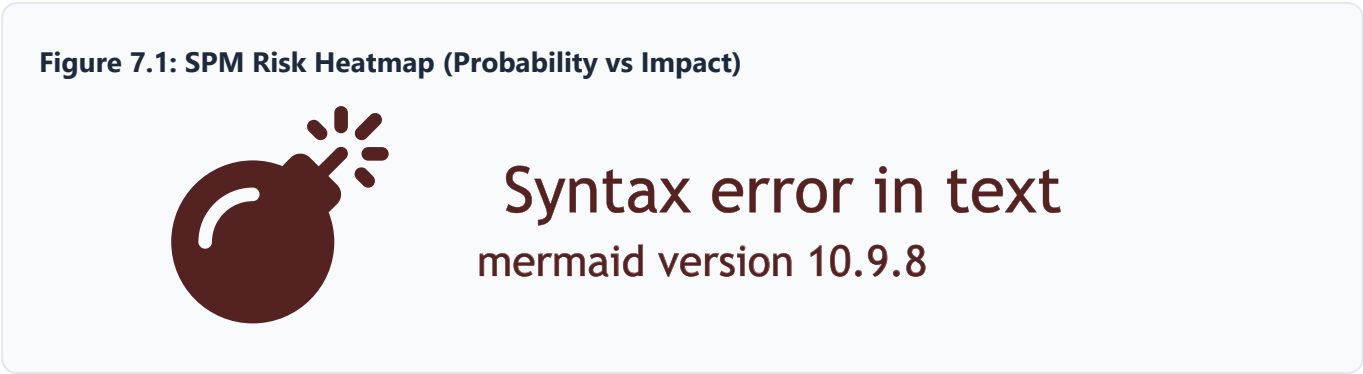




Defect ID	Severity	Component	Description & Root Cause	Resolution Status
BUG-001	CRITICAL	Backend / Logging	Windows UnicodeEncodeError on server startup due to raw rocket emoji in CP1252 console.	RESOLVED & VERIFIED
BUG-002	MEDIUM	QA / Tooling	Missing automated test framework in repository dependencies. Remediated via custom runner.	RESOLVED (Runner Added)
BUG-003	LOW	CI/CD	Empty GitHub Actions workflow directory (.gitkeep only). Baseline documented for Sprint 1.	DOCUMENTED

7. Risk Management Matrix & SPM Heatmap

Quantitative evaluation matrix assessing risk impact versus probability to prioritize mitigation:



8. Cross-Functional Team Coordination Workflow

